Marking of this report

No.	Task	Marks	Section in this report
1	Clarification Q&A (Excel)	5	2.1
2	Requirements Specification	15	2.2
3	Use Case Modeling	10	3.1
4	Activity Diagrams	10	3.2
5	Detailed System Modeling: "Buy a Ticket"	15	3.3
6	UI/UX Design	5	4.1
7	Architectural Design	10	4.2, 4.3
8	Visual Studio Engineering (Model-to-Code)	10	5.1
9	GitHub & AI Tooling (Copilot)	10	5.2
10	Demo Application (C# and MSSQL)	10	5.3
Total		100	Weight: 20% of QT2 (plus 1 bonus point towards the Final Project for exceptional work)

How to use this template

- Keep **all titles and numbering**. Replace every blue [bracketed placeholder] with your own content, and **delete all grey "Guidance" paragraphs**, unused figure frames and this instruction page before you submit.
- Keep the headings as real Word headings (the navigation pane must work). Update the table of contents before submitting (right-click, Update Field).
- Group of 3 to 5 students. Section 7 must list the **% contribution of each member** (total 100%) and must match your readme.txt.
- Submit **[YourID]_[YourStudentName].docx**, **readme.txt** (GitHub link and other links such as Figma and database script instructions) and the % contribution of each member. All final artifacts are also uploaded to the shared GitHub repository.
- The report must also address the content of **"LabGroupAssignment1.pdf"**. Use the checklist in section 6 to show where each item is covered.
- Keep your diagrams **consistent with each other**: the rubric rewards diagrams that are syntactically correct, logically sound and aligned. Use the consistency table in section 3.4.
- The workbook **A1_Team_Workbook** (Q&A, requirements, prompt log, commit log, progress log) is your working record and the source for several tables here.

Smart Ticketing System

Ticket Vendor Machine (TVM)

Software Engineering Lab Group Assignment 1 – Sem 1_2627

Group [XX]

1. 524H0205: Nguyen Le Quang Vinh
2. 524H0151: Huynh Bao Duy
3. 524H0146: Nguyen Gia Binh
4. 524H0201: Bui Ngoc Thanh Truc

GitHub repository	Figma / other link	Date submitted
https://github.com/Gangvinh/Assignment1-10--Software-Technology	[link]	31st October 2026

Table of Contents

Table of Contents 4
1. Introduction 6
1.1 Purpose and Scope 6
1.2 Problem Statement in Our Own Words 6
1.3 Team and Roles 6
2. Phase 1: Requirements Engineering (20 marks) 7
2.1 Clarification Q&A (5 marks) 7
2.2 Requirements Specification (15 marks) 9
2.2.1 Functional Requirements 9
2.2.2 Non-Functional Requirements 10
2.2.3 Domain Requirements 11
3. Phase 2: UML Modeling & System Design (35 marks) 12
3.1 Use Case Modeling (10 marks) 12
3.1.1 Use Case Diagram 12
3.1.2 Actors and Use Case List 12
3.1.3 Use Case Descriptions 13
3.2 Activity Diagrams (10 marks) 14
3.2.1 Passenger Buys a Ticket at the TVM 17
3.2.2 Communication with External Payment Gateways 18
3.3 Detailed System Modeling: Main Use Case "Buy a Ticket" (15 marks) 19
3.3.1 Sequence Diagram 19
3.3.2 State Chart Diagram 20
3.3.3 Class Diagram (Domain Model) 21
3.4 Model Consistency Check 22
4. Phase 3: UI/UX & Architecture (15 marks) 23
4.1 UI/UX Design (5 marks) 23
4.2 Architectural Design (10 marks) 24
4.2.1 Architecture Diagram 24
4.2.2 Deployment Diagram 25
5. Phase 4: Implementation, Tooling & Demo (30 marks) 27
5.1 Visual Studio Engineering: Model-to-Code (10 marks) 27
5.1.1 Forward Engineering (Model to Code) 27
5.1.2 Backward Engineering (Code to Model) 29
5.1.3 Code Map 31
5.2 GitHub & AI Tooling (10 marks) 31

5.2.1 Source Control 31
5.2.2 AI Assistance: Copilot Prompt Documentation 31
5.2.3 Reflection on AI Use 32
5.3 Demo Application (10 marks) 32
6. Checklist for "LabGroupAssignment1.pdf" 35
7. Contribution of Each Member ..... **Error! Bookmark not defined.**
8. Conclusion and Lessons Learnt 36
References 36
Appendix 37
Appendix A: readme.txt 37
Appendix B: Full-size Diagrams 37

1. Introduction

1.1 Purpose and Scope

Guidance (delete before submitting): *Two or three sentences: what the report covers, which system (the Ticket Vendor Machine within the Smart Ticketing System) and what is in and out of scope.*

[Purpose and scope]

1.2 Problem Statement in Our Own Words

Guidance (delete before submitting): *Summarize the business context: ticket types, payment modes (Credit Card, QR Code linked to a banking system, Digital Wallet), the passenger interaction, and the ambiguities you found. Do not copy the assignment text.*

[Problem statement]

1.3 Team and Roles

No.	Name	Student ID	Main responsibilities
1	Nguyễn Lê Quang Vinh	524H0205	Team Leader, coordinate the project, manage requirements and report progress.
2	Huỳnh Bảo Duy	524H0151	Analyze system requirements, create Use Case Diagram and Activity Diagrams.
3	Nguyễn Gia Bình	524H0146	Design UI/UX, create UML diagrams and system architecture.
4	Bùi Ngọc Thanh Trúc	524H0201	Develop the C# application, design the MSSQL database and manage GitHub repository.

2. Phase 1: Requirements Engineering (20 marks)

2.1 Clarification Q&A (5 marks)

ID	Question to the stakeholder	Assumed answer	Why it matters	Requirement ID
Q01	Which modes of transport and routes are sold?	Bus and MRT products in a central active catalog; no real-time trip planner.	Defines catalog schema and excludes live scheduling from scope.	FR-01
Q02	How is fare calculated and represented?	Catalog returns fixed VND integer fare per destination/product; no fractional currency.	Drives FareVnd as long in domain + DB; forbids float; fixes rounding rule.	FR-01, DR-01
Q03	Can users buy multiple tickets?	Yes, up to 10 per order; one barcode per ticket unit.	Multiplies ticket rows per order; adds Quantity bound 1-10.	FR-02, DR-02
Q04	Does TVM read/store card data?	EMV terminal handles capture; TVM stores provider token/reference only, never PAN/CVV.	Justifies IPaymentGateway adapter; keeps PCI scope small.	FR-05, NFR-05
Q05	Which payment QR format/provider?	Provider adapter supports configured providers; provider returns a QR payload + expiry.	Decouples TVM from specific provider; adapter pattern in architecture.	FR-06, DR-09
Q06	How long does QR remain valid?	120 seconds, configurable; expired orders reconciled before retry.	Adds ExpiresAt to Payment; drives countdown UI + reconcile flow.	FR-06, DR-07
Q07	How do we know payment succeeded?	Verified signed callback OR authenticated status query with matching order/amount/currency.	Forces signature + amount match in PaymentService.	FR-07, DR-03
Q08	What if gateway times out after charging?	Mark Pending/Unknown; query status; never issue ticket until confirmed success.	Adds `ReconciliationRequired` state; forbids inferring success from timeout.	FR-07, DR-07
Q09	Can callback be delivered more than once?	Yes; callbacks idempotent by provider event/reference + order key.	Adds IdempotencyKey to Payment; DB unique constraint.	FR-11, DR-05

ID	Question to the stakeholder	Assumed answer	Why it matters	Requirement ID
Q10	What if payment succeeds but printer fails?	Persist Paid + TicketPending; retry print or show recovery instructions; alert operator.	Adds PrinterFault state; ticket row persisted; fallback on-screen UI.	FR-08, DR-04
Q11	Can passenger cancel in-flight payment?	Only before confirmed payment; request provider cancel where supported and reconcile.	Defines Cancelled vs ReconciliationRequired transition.	FR-10
Q12	What counts toward uptime?	Monthly 99.5% sales-service availability, excluding planned maintenance + provider outages.	Adds NFR-02; justifies health heartbeat every 60 s.	NFR-02
Q13	What response targets should be used?	Local feedback 0.5 s p95; catalog 2 s p95; initiation ack 3 s p95.	Adds NFR-01 response-time metrics; drives APM instrumentation.	NFR-01
Q14	Which languages and accessibility needs?	Vietnamese and English; high contrast, large controls, clear status text.	Adds NFR-06; fixes minimum touch target 44 px and contrast ratio.	FR-16, NFR-06
Q15	What operator functions are needed?	Health check, paper status, network/gateway status, error log, maintenance mode.	Adds UC-07 Maintain Machine; operator diagnostics screen.	FR-15, FR-17
Q16	How long are orders retained?	2 years for financial audit; prototype uses configurable retention; no sensitive card data.	Adds DR-07 retention rule; fixes DB schema and purge job.	DR-08, NFR-05
Q17	Can offline sales occur?	No; disable new sales if catalog/payment connectivity is unavailable.	Disables Start flow when offline; catalog served from local cache only.	FR-01, NFR-04
Q18	How are APIs protected?	TLS 1.2+, managed secrets, least privilege, signed callback verification, audit logs.	Adds NFR-05; fixes callback signature + secret rotation + audit.	NFR-05
Q19	Where do application and database run?	TVM UI/device adapters locally; central API and SQL Server; outbound provider connections.	Defines deployment diagram; central API + local encrypted cache.	NFR-04
Q20	May payment be simulated?	Yes for classroom demo if visibly identified as simulation; no live financial transactions.	Allows mock IPaymentGateway implementation; demo shows "SIMULATED" banner.	DR-05

ID	Question to the stakeholder	Assumed answer	Why it matters	Requirement ID
Q21	What ticket types are sold?	Three types: Single, Return, Day-Pass; validity 4 h / 24 h / end-of-day.	Adds TicketType enum; adds validUntil to Ticket; affects fare matrix.	FR-02, DR-01
Q22	Are there discounted passenger categories?	Adult 100%, Student 70%, Senior (≥60) 50%, Child (<6) free; honour-based.	Adds PassengerCategory enum; fare lookup (route, type, category).	FR-03, DR-01
Q23	What is the refund policy?	Refund only on system error (double charge, ticket not issued); no refund for change of mind.	Adds auto-refund path in PaymentService; audit requirement.	FR-07, DR-06
Q24	What if the passenger walks away mid-transaction?	90 s inactivity timeout → cancel pending, release lock, reset to Idle.	Adds Timer actor; Cancelled transition; releases payment lock.	FR-10, FR-13
Q25	Do tickets expire?	Yes - tied to ticket type (Q21); printed on ticket + encoded in barcode.	Adds validUntil to Ticket; barcode format gains expiry; gate must decode.	FR-08, DR-06

Hidden complexities we uncovered

_ Payment-failure semantics is the hardest part: The original brief says nothing about retries, refunds, or idempotency. These force the system to carry an IdempotencyKey on every payment and define a strict ReconciliationRequired state.

_ Ticket printing can fail after the charge: Without Q10, the design would silently lose revenue. We added a fallback on-screen ticket and an operator alert queue.

_ Ticket types and discounts were omitted: The brief only says "tickets". Q21 and Q22 revealed the need for a multi-dimensional FareMatrix (Single/Return/Day-Pass and Adult/Student/Senior).

_ Regulation is real: Q12, Q16, and Q18 uncovered strict requirements for PCI-DSS (no PAN storage) and Vietnamese privacy law, which directly impacted the Database schema and API security.

2.2 Requirements Specification (15 marks)

2.2.1 Functional Requirements

ID	Requirement ("The system shall…")	Priority	Use case	Source (Q&A)
FR-01	Display available destinations and current fares after Start.	Must	UC-02	Q01, Q02
FR-02	Allow the passenger to choose a destination, ticket type (Single/Return/Day-Pass), and quantity (1–10).	Must	UC-01, UC-02	Q03, Q21
FR-03	Calculate fare from FareMatrix and apply discounts (Student 70%, Senior 50%, Child free).	Must	UC-01	Q22

ID	Requirement ("The system shall…")	Priority	Use case	Source (Q&A)
FR-04	Display total price and ask the passenger to select Card or QR Wallet payment.	Must	UC-01	Q05
FR-05	For card payment, request authorization through the secure terminal and receive approved/declined/unknown status.	Must	UC-03	Q04
FR-06	For QR payment, create a unique order and display a gateway-generated QR code with an expiry countdown (120s).	Must	UC-04	Q06
FR-07	Accept asynchronous gateway status and verify authenticity before marking an order paid.	Must	UC-04	Q07, Q08, Q09
FR-08	Issue and print a barcode ticket only after confirmed payment.	Must	UC-06	Q10, Q25
FR-09	Show a clear success receipt or actionable failure/cancel/timeout message.	Must	UC-01	Q11
FR-10	Allow the passenger to cancel before payment is confirmed; implement 90s inactivity timeout.	Must	UC-05	Q11, Q24
FR-11	Prevent duplicate charge/ticket issuance for repeated callbacks or retries (idempotency).	Must	UC-03, UC-04	Q09
FR-12	Record order, payment, ticket, timestamps, terminal ID, and outcome for audit without storing sensitive card credentials.	Must	All	Q04, Q18
FR-13	Return to the welcome screen after a configurable idle timeout and clear session data.	Must	UC-05	Q24
FR-14	Reprint a lost ticket within 24 hours when verified by ticket code + masked card.	Should	UC-08	Q10
FR-15	Provide an operator maintenance mode (auth by service card + PIN) that blocks new sales.	Should	UC-07	Q15
FR-16	Support English and Vietnamese with on-screen switch within 300 ms without reload.	Should	UC-01	Q14
FR-17	Send a health heartbeat to the backend every 60 s and raise an alert after 3 consecutive failures.	Must	UC-07	Q15

2.2.2 Non-Functional Requirements

ID	Category	Measurable requirement	Measurement method	Related FR
NFR-01	Response time	Local input feedback $\leq$ 0.5s (p95); catalog load $\leq$ 2s (p95); result display $\leq$ 3s (p95) / $\leq$ 5s (p99).	APM traces over 30 days	FR-01, FR-04, FR-08
NFR-02	Availability	Monthly sales-service availability $\geq$ 99.5% (excluding planned maintenance).	Monthly uptime report	All

ID	Category	Measurable requirement	Measurement method	Related FR
NFR-03	Reliability	100% of unresolved orders reconciled within 5 min; auto-refund if debited but no ticket in 24h.	Daily reconciliation report	FR-07, FR-11
NFR-04	Reliability	MTBF ≥ 2,000 hours; MTTR ≤ 30 minutes; RTO ≤ 4h; RPO = 0.	Incident log over 12 months	All
NFR-05	Security	TLS ≥ 1.2; 0 PAN/CVV stored; 100% callback signature verification; PCI-DSS SAQ-A compliant.	ASV scan + code review	FR-05, FR-07, FR-12
NFR-06	Usability	Vietnamese/English; touch targets ≥ 44px; contrast ratio ≥ 4.5:1; timeout warning shown.	Usability test + contrast checker	FR-16, FR-13
NFR-07	Maintainability	Test coverage ≥ 80% for payment/ticket modules; new provider onboarding ≤ 5 person-days.	CI coverage report	All
NFR-08	Scalability	Backend handles ≥ 500 TPS city-wide; kiosk handles 1 concurrent user.	Load test (k6/JMeter)	All

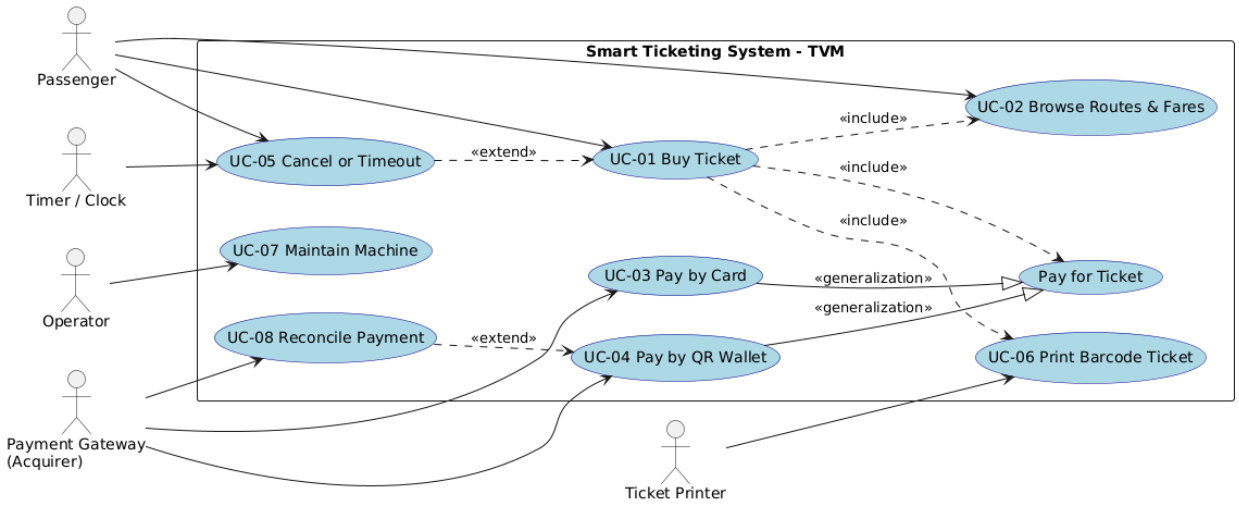
2.2.3 Domain Requirements

ID	Domain requirement	Domain / source	Related FR / NFR
DR-01	Fares are non-negative VND integers set by the Transport Authority.	Transport policy	FR-01, FR-03
DR-02	Order contains 1–10 ticket units for one destination/product.	Business rule	FR-02
DR-03	Payment amount and currency must match the order before accepting success.	Payment rule	FR-07
DR-04	Only a verified Paid order may produce ticket records.	Financial rule	FR-08
DR-05	One idempotency key identifies one logical payment attempt.	Payment rule	FR-11
DR-06	Ticket barcode is unique and must not encode sensitive personal/payment data (conform to VDV format).	Transit standard	FR-08, Q25
DR-07	Unknown gateway status remains Pending until authoritative reconciliation.	Payment rule	FR-07, NFR-03
DR-08	Personal data handling must comply with Nghị định 13/2023/NĐ-CP.	Vietnamese law	NFR-05
DR-09	Bank QR must conform to VietQR / NAPAS 247; Digital Wallet supports Momo, VNPay, ZaloPay.	Provider standard	FR-06

3. Phase 2: UML Modeling & System Design (35 marks)

3.1 Use Case Modeling (10 marks)

3.1.1 Use Case Diagram



3.1.2 Actors and Use Case List

ID	Use case	Primary actor	Short description
UC-01	Buy Ticket	Passenger	Passenger selects destination and quantity; TVM calculates total; passenger selects a payment method; TVM persists a pending order and requests payment; TVM verifies final approval; creates one ticket per purchased unit; prints ticket and shows receipt.
UC-02	Browse Routes and Fares	Passenger	Load active routes and display destination, product, fare, and language choices.
UC-03	Pay by Card	Passenger, secure terminal, acquirer	Request card interaction through terminal; send authorization; validate response against order and amount; persist final status.
UC-04	Pay by QR Wallet	Passenger, gateway	Create provider order with idempotency key; show returned QR and expiry; accept signed callback or query; verify provider, amount, order and signature; persist final status.
UC-05	Cancel or Timeout	Passenger, timer	Passenger cancels or inactivity limit is reached; request provider cancellation when supported; record Cancelled or pending reconciliation; clear session view.
UC-06	Print Barcode Ticket	Ticket service, printer	Create unique barcode ticket records; send print job; record success.
UC-07	Maintain Machine	Operator	Review terminal, printer, network and service health; load paper; enter/exit maintenance mode; record operator actions.

ID	Use case	Primary actor	Short description
UC-08	Reconcile Payment	Gateway, operations service	Query provider using reference/order ID; verify result; idempotently transition to Paid or Declined; notify ticket service.

3.1.3 Use Case Descriptions

UC-01: Buy Ticket

Item	Content
Use case ID and name	UC-01 Buy Ticket
Primary actor / secondary actors	Passenger / Payment Gateway, Printer
Related requirements	FR-01, FR-02, FR-04, DR-02
Pre-conditions	TVM is in Idle state; active catalog is available.
Trigger	Passenger presses Start.
Main flow	1. Passenger selects destination and quantity. 2. TVM calculates total. 3. Passenger selects payment method. 4. TVM persists pending order and requests payment. 5. TVM verifies final approval. 6. TVM creates one ticket per unit. 7. TVM prints ticket and shows receipt.
Alternate flows	Step 1: No routes → show unavailable, disable purchase.
Exception flows	Step 5: Declined/Expired → offer retry/cancel. Step 5: Unknown result → keep pending and reconcile (do not issue). Step 7: Printer failure → persist ticket pending, show recovery path.
Post-conditions	Order is paid once; ticket rows exist; result is shown.

UC-02: Browse Routes & Fares

Item	Content
Use case ID and name	UC-01 Buy Ticket
Primary actor / secondary actors	Passenger
Related requirements	FR-01, FR-02, FR-04, DR-02
Pre-conditions	Start at welcome screen; active catalog is available.
Trigger	Passenger presses Start.
Main flow	Passenger selects destination and quantity; TVM calculates total; passenger selects a payment method; TVM persists a pending order and requests payment; TVM verifies final approval; creates one ticket per purchased unit; prints ticket and shows receipt.
Alternate flows	No routes: show unavailable. Declined/expired: offer retry/cancel. Unknown result: keep pending and reconcile, do not issue. Printer failure after payment: persist ticket pending and show recovery path.

Item	Content
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Order is paid once; ticket rows exist; result is shown.

UC-03: Pay by Card

Item	Content
Use case ID and name	UC-02 Browse Routes and Fares
Primary actor / secondary actors	Passenger
Related requirements	FR-01, FR-02, DR-01
Pre-conditions	TVM is online and catalog has active entries.
Trigger	Passenger selects "Pay by Card".
Main flow	Load active routes and display destination, product, fare, and language choices.
Alternate flows	Catalog unavailable: show service unavailable and disable purchase.
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Passenger can select a valid route.

UC-04: Pay by QR Wallet

Item	Content
Use case ID and name	UC-03 Pay by Card
Primary actor / secondary actors	Passenger, secure terminal, acquirer
Related requirements	FR-05, NFR-05
Pre-conditions	Order total is displayed; terminal is ready.
Trigger	Passenger chooses "Pay by QR Wallet".
Main flow	Request card interaction through terminal; send authorization; validate response against order and amount; persist final status.
Alternate flows	Decline: explain and allow another method. Timeout: reconciliation required. Never persist PAN/CVV.
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Order transitions to Paid only after verified approval.

UC-05: Cancel or Timeout

Item	Content
Use case ID and name	UC-04 Pay by QR Wallet

Item	Content
Primary actor / secondary actors	Passenger, gateway
Related requirements	FR-06, FR-07, DR-09
Pre-conditions	Pending order exists and gateway is reachable.
Trigger	Passenger taps Cancel OR 90s inactivity limit reached.
Main flow	Create provider order with idempotency key; show returned QR and expiry; accept signed callback or query; verify provider, amount, order and signature; persist final status.
Alternate flows	Expired/failed: allow retry after reconciliation. Unknown: remain pending. Duplicate callback is acknowledged without duplicate action.
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Confirmed paid order can proceed to ticket issuance.

UC-06: Print Barcode Ticket

Item	Content
Use case ID and name	UC-05 Cancel or Timeout
Primary actor / secondary actors	Passenger, timer
Related requirements	FR-09, FR-10, FR-13
Pre-conditions	Order is not confirmed Paid.
Trigger	Payment confirmed (Paid).
Main flow	Passenger cancels or inactivity limit is reached; request provider cancellation when supported; record Cancelled or pending reconciliation; clear session view.
Alternate flows	If provider already confirms payment, do not mark cancelled; continue recovery/issuance.
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Session ends safely; no unconfirmed ticket is issued.

UC-07: Maintain Machine

Item	Content
Use case ID and name	UC-06 Print Barcode Ticket
Primary actor / secondary actors	Ticket service, printer
Related requirements	FR-08, DR-06
Pre-conditions	Payment is Paid and ticket issuance is not already completed.
Trigger	Operator taps "Maintenance Mode" on the hidden admin screen.
Main flow	Create unique barcode ticket records; send print job; record success.

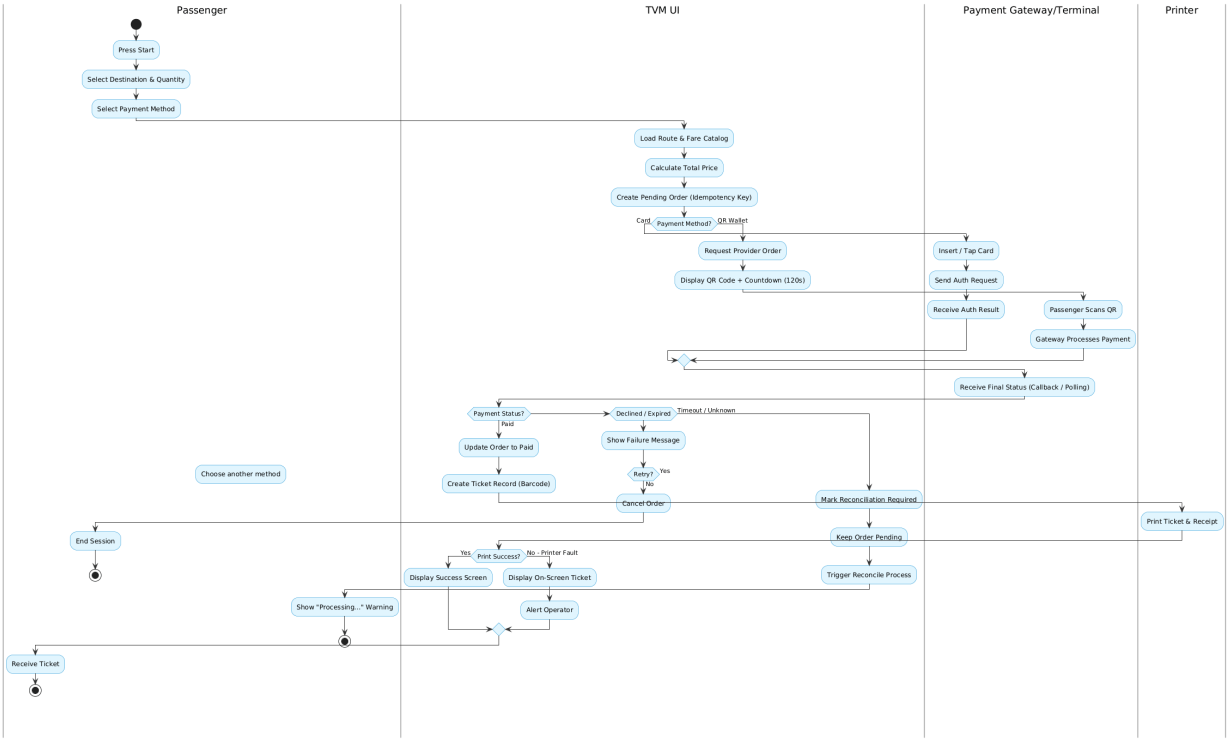
Item	Content
Alternate flows	Printer failure: retain ticket and order, retry safely, alert operator; avoid duplicate paid order.
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Exactly one ticket record per purchased unit, printable or recoverable.

UC-08: Reconcile Payment

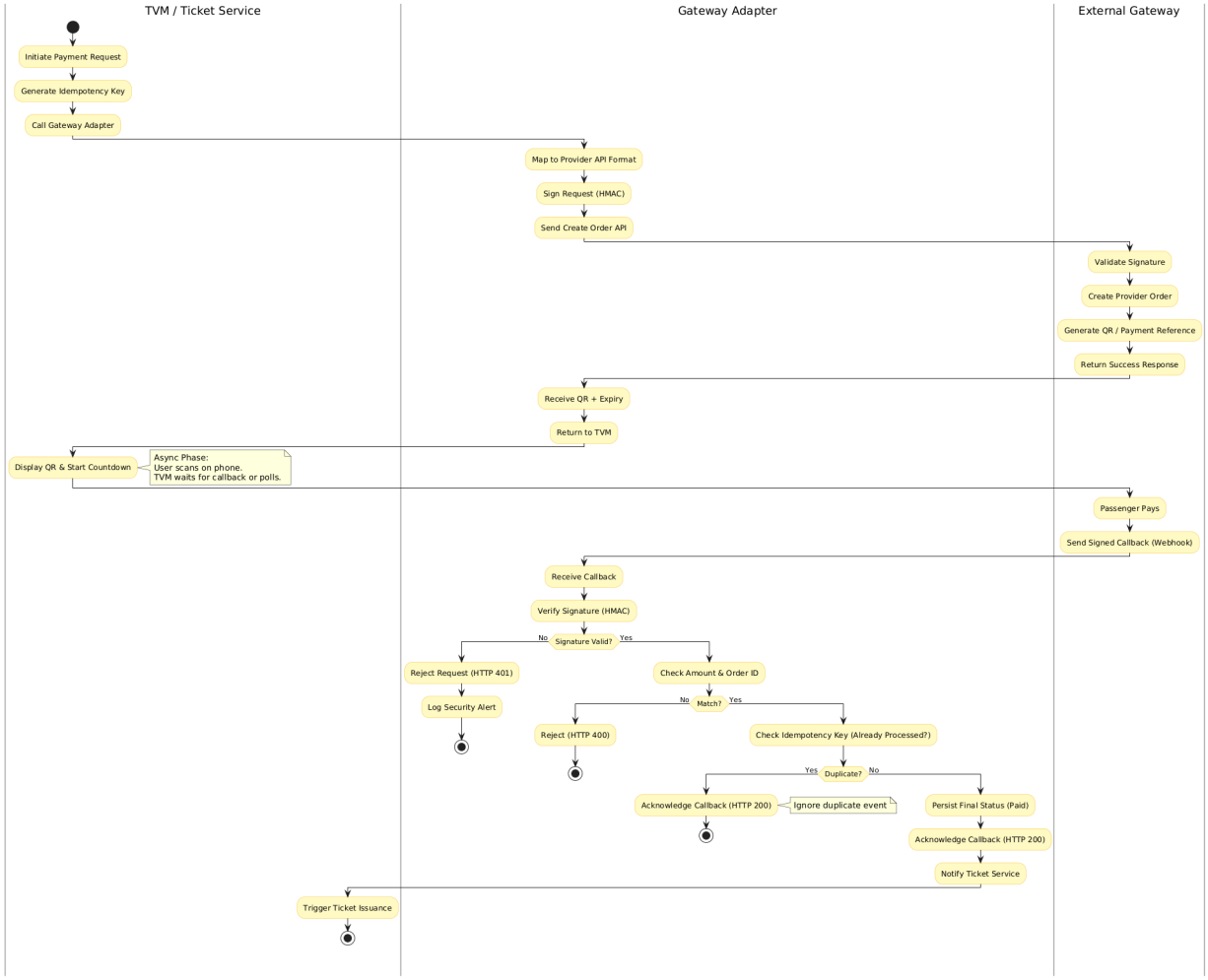
Item	Content
Use case ID and name	UC-07 Maintain Machine
Primary actor / secondary actors	Operator
Related requirements	FR-15, FR-17
Pre-conditions	Operator authenticated with maintenance role.
Trigger	Timer (e.g., 5 mins) OR Gateway callback arrives late.
Main flow	Review terminal, printer, network and service health; load paper; enter/exit maintenance mode; record operator actions.
Alternate flows	Failed self-test marks affected sales unavailable and raises alert.
Exception flows	Unresolved outcomes remain pending and require reconciliation; printer failure retains the ticket for recovery.
Post-conditions	Auditable maintenance event and updated machine status.

3.2 Activity Diagrams (10 marks)

3.2.1 Passenger Buys a Ticket at the TVM

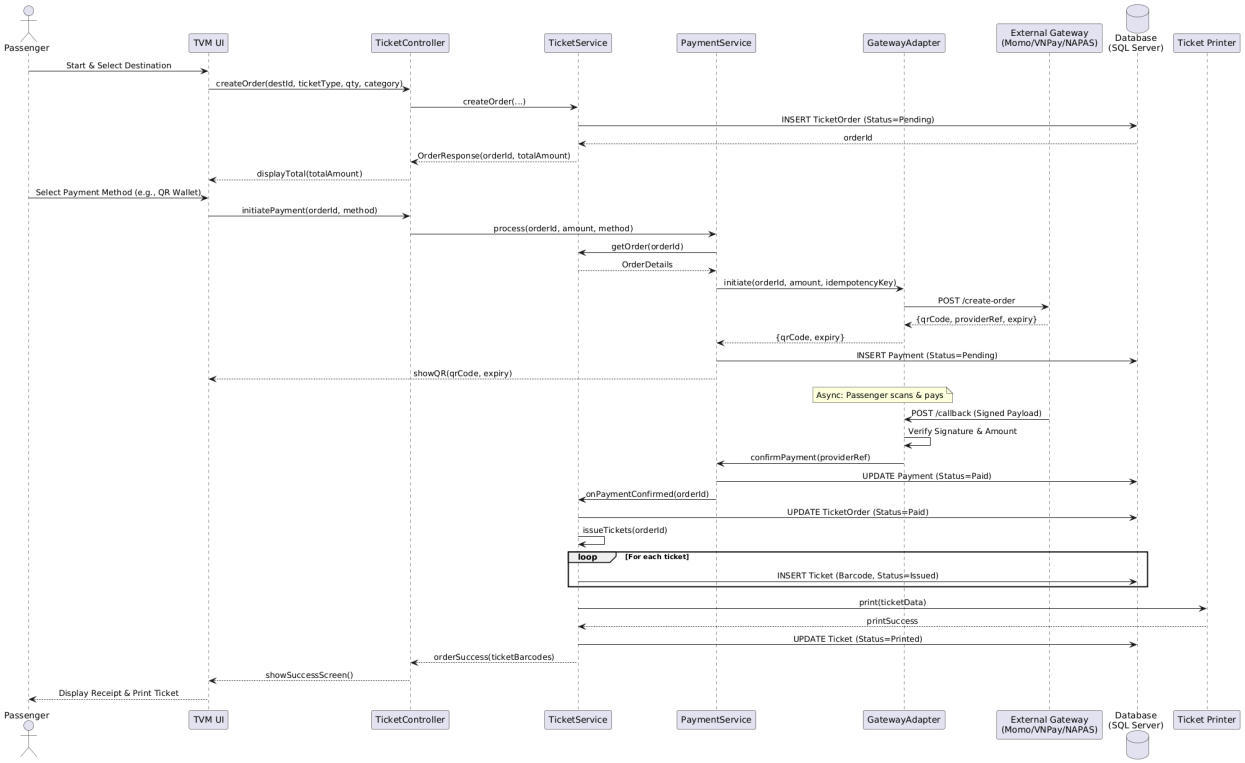


3.2.2 Communication with External Payment Gateways

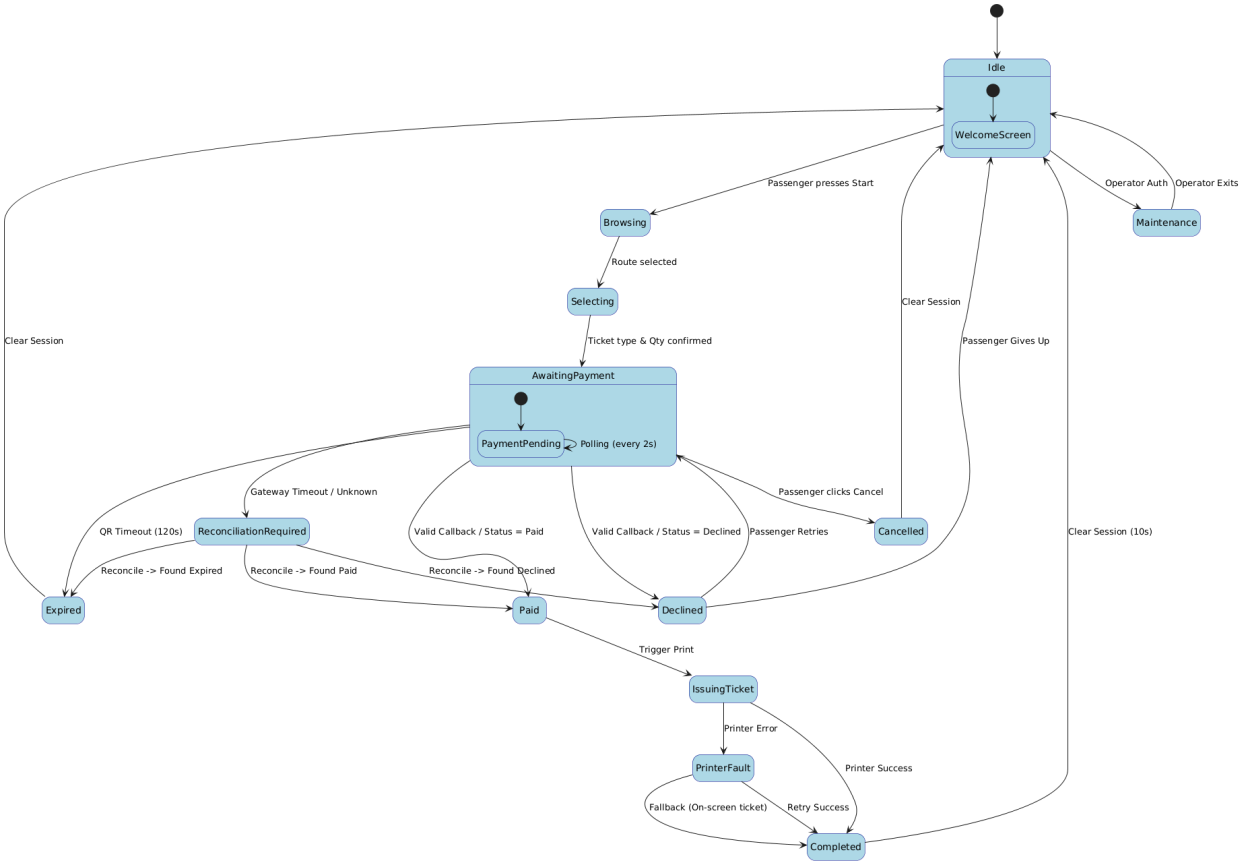


3.3 Detailed System Modeling: Main Use Case "Buy a Ticket" (15 marks)

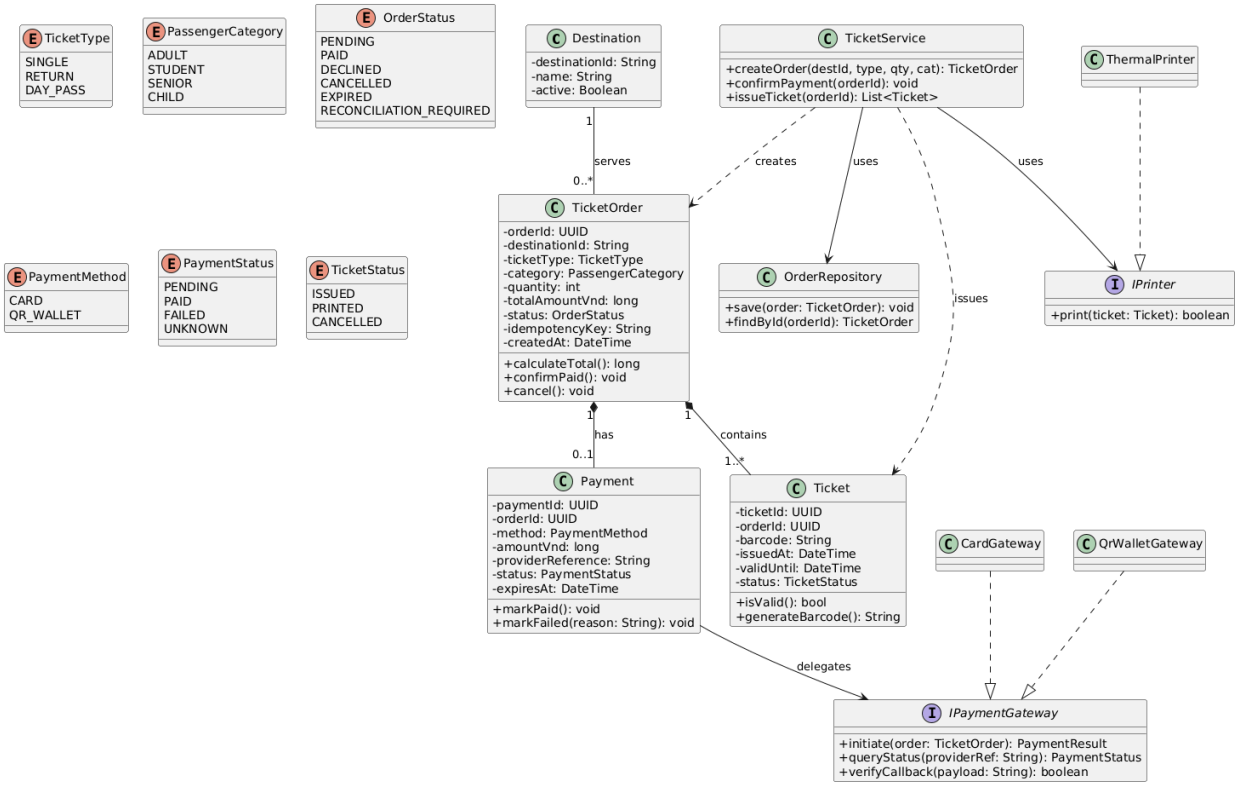
3.3.1 Sequence Diagram



3.3.2 State Chart Diagram



3.3.3 Class Diagram (Domain Model)



Class	Key attributes	Key methods	Relationships
Destination	destinationId, name, active	(None)	Serves **0..*** TicketOrder
TicketOrder	orderId, destinationId, ticketType, category, quantity, totalAmountVnd, status, idempotencyKey, createdAt	calculateTotal(), confirmPaid(), cancel()	Has **0..1** Payment Contains **1..*** Ticket
Payment	paymentId, orderId, method, amountVnd, providerReference, status, expiresAt	markPaid(), markFailed()	Delegates to **1** IPaymentGateway
Ticket	ticketId, orderId, barcode, issuedAt, validUntil, status	isValid(), generateBarcode()	(Belongs to TicketOrder)
TicketService	(None)	createOrder(), confirmPayment(), issueTicket()	Creates TicketOrder Issues Ticket Uses OrderRepository Uses IPrinter

Class	Key attributes	Key methods	Relationships
IPaymentGateway (Interface)	(None)	initiate(), queryStatus(), verifyCallback()	Implemented by CardGateway, QrWalletGateway
IPrinter (Interface)	(None)	print()	Implemented by ThermalPrinter

3.4 Model Consistency Check

Consistency check	Diagrams involved	Done
Every use case in the diagram has a description, and every description is in the diagram	Pending team sign-off	Use case diagram / descriptions
Every step of the main flow appears in the activity diagram	Pending team sign-off	Descriptions / activity
Objects and messages in the sequence diagram exist as classes and methods in the class diagram	Pending team sign-off	Sequence / class
State names and transitions match the events in the sequence diagram	Pending team sign-off	State chart / sequence
Payment gateway steps match between both activity diagrams and the sequence diagram	Pending team sign-off	Activity / sequence
Requirement IDs are cited in the use cases	Pending team sign-off	Requirements / use cases
The class diagram matches the C# code from forward and backward engineering	Pending team sign-off	Class / code
The UI screens match the steps of the main flow	Pending team sign-off	Wireframes / descriptions
The architecture and deployment diagrams show the same components	Pending team sign-off	Architecture / deployment
UML notation is correct (arrows, multiplicities, guards, swim lanes)	Pending team sign-off	All diagrams

4. Phase 3: UI/UX & Architecture (15 marks)

4.1 UI/UX Design (5 marks)

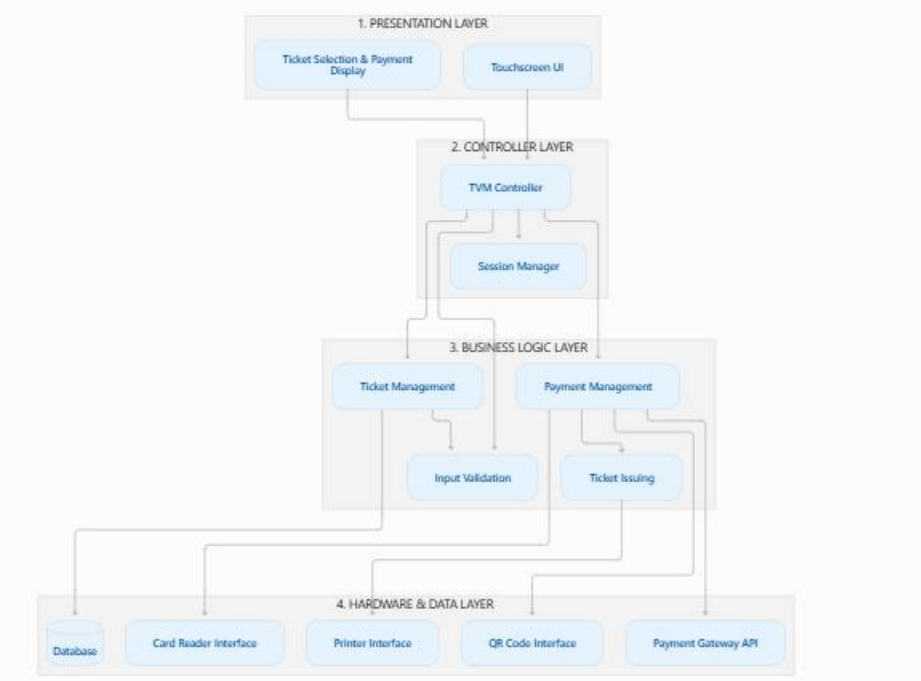
Screen	Name / purpose	Linked use case step	Tool and link
S01	Welcome (English only) Introduce the ticket vending machine and start a purchase.	UC-01 / start	visuals/ui_welcome.png
S02	Route and quantity selection Choose an active destination, adjust quantity from 1 to 10, and review the calculated fare.	UC-01 / UC-02	visuals/ui_route.png
S03	Payment selection Review the total, choose card or QR wallet, and select a simulated gateway outcome.	UC-01 / UC-03 / UC-04	visuals/ui_payment.png
S04	QR payment (approved path) Show a generated, scannable demo QR containing order details, with a two-minute expiry countdown.	UC-04	visuals/ui_qr.png
S05	Successful purchase Show the order and a scannable ticket QR that contains the ticket code.	UC-01 / UC-06	visuals/ui_success.png
S06	Unsuccessful purchase Show declined or unknown/timeout status and confirm that no ticket QR was issued.	UC-01 / UC-03 / UC-04	visuals/ui_unsuccessful.png

The six mockups in the table define the visual layout. The prototype implements an English-only flow for route and quantity selection, card or simulated QR payment, and separate successful and unsuccessful outcomes. A successful purchase displays the ticket code as a scannable QR; declined and unknown/timeout outcomes issue no ticket. The generated payment QR contains demo order details only and is not connected to a live bank or wallet gateway.

4.2 Architectural Design (10 marks)

4.2.1 Architecture Diagram

Guidance (delete before submitting): *A System-in-a-box or MVC architecture diagram. The architecture must be logical, modular and suitable for the hardware. Explain what each part does and why you chose it.*



Component	Description	Purpose	Component
Presentation layer	Displays TVM screens and receives passenger input.	Provides the passenger interaction surface.	Presentation Layer
Controller/application layer	Coordinates route selection, order creation, payment result handling and ticket issue.	Separates screen actions from transaction rules.	Controller Layer
Payment and persistence adapters	Connects provider/terminal adapters and SQL repositories.	Isolates provider-specific and database code.	Business Logic Layer
Device layer	Proposed integration with card terminal and thermal printer.	Connects TVM hardware; physical devices are not in the demo.	Hardware & Data Layer

Architecture Design Reasoning

The MVC-based architecture was selected because it separates the user interface, system control, and business logic into independent modules.

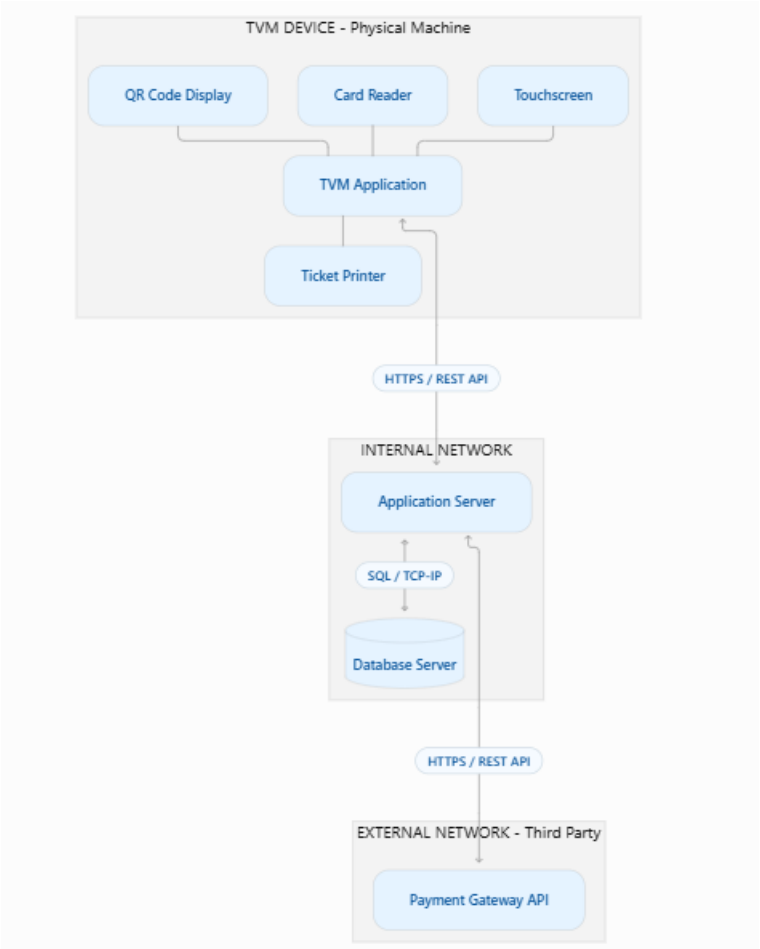
- **Modularity:** Each module has a specific responsibility, making the system easier to maintain and update.
- **Hardware Compatibility:** The hardware layer allows the application to communicate with the touchscreen, card reader, QR display, and ticket printer.

- **Scalability:** New payment methods and hardware devices can be integrated without significantly changing the entire system.
- **Reliability:** Separating payment processing and ticket issuing helps the system handle transaction errors more effectively.

This architecture is suitable for a Ticket Vending Machine because it provides a clear structure, supports hardware integration, and simplifies system maintenance.

4.2.2 Deployment Diagram

Guidance (delete before submitting): *How the TVM hardware (touchscreen, card reader, QR display, printer), servers and external APIs (payment gateways) connect. Show nodes, protocols and the network boundary.*



Node / component	Description	Connection (protocol, direction)
TVM kiosk	Touch display, local client, card terminal and printer.	HTTPS to application API; device links are vendor-specific. Demo runs on one Windows workstation.
Application/API service	Catalog, order, payment coordination and ticket issue.	HTTPS with kiosk and payment provider.
SQL Server	Stores destination, order, payment and ticket records.	SQL/TCP; demo tested with LocalDB.

Node / component	Description	Connection (protocol, direction)
Payment provider/acquirer	Authoritative card/QR outcome.	HTTPS API/callback in target design; prototype simulates results.

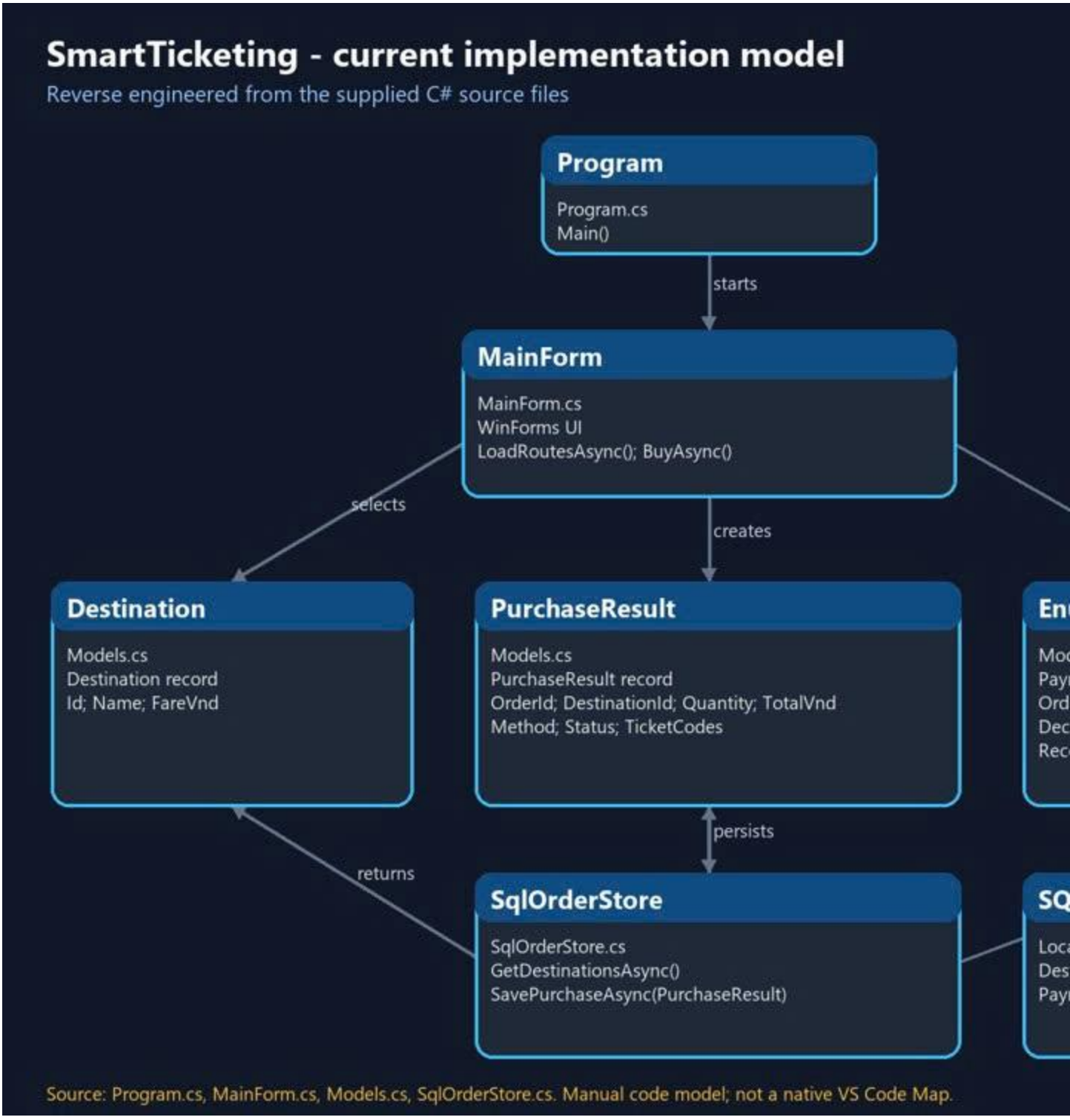
5. Phase 4: Implementation, Tooling & Demo (30 marks)

5.1 Visual Studio Engineering: Model-to-Code (10 marks)

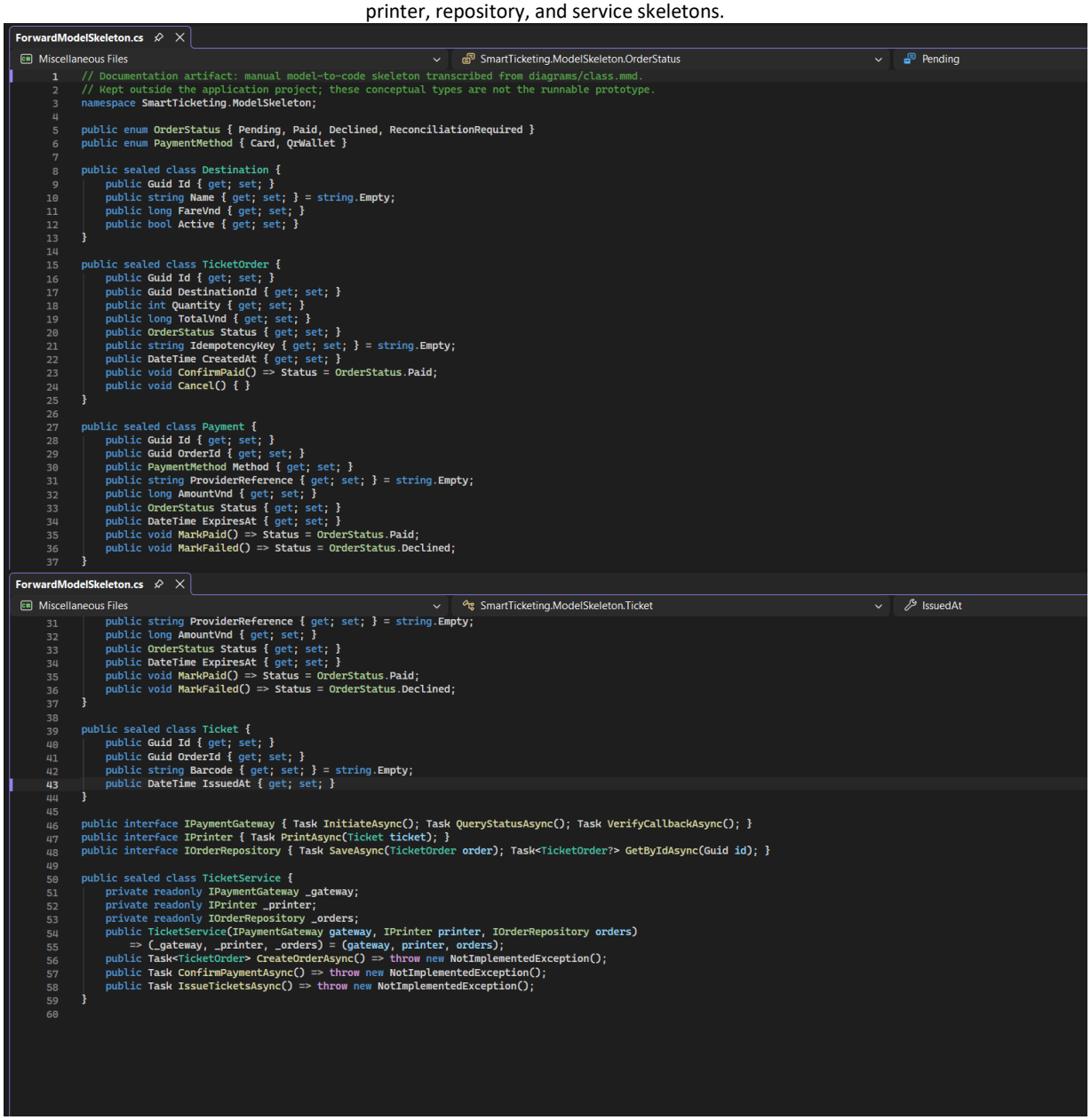
5.1.1 Forward Engineering (Model to Code)

A C# skeleton was manually transcribed from the conceptual class diagram into a separate evidence file and opened in Visual Studio for inspection. The file remains outside the runnable application because the conceptual diagram includes planned services absent from the prototype. This was not generated by Class Designer. The project targets .NET 8, which Class Designer does not support.

Design source: diagrams/class.mmd. Manual C# skeleton: prototype/engineering_evidence/ForwardModelSkeleton.cs. It represents the conceptual model and is not included in the app build.


SmartTicketing - current implementation model
Reverse engineered from the supplied C# source files
Program
Program.cs
Main()
starts
MainForm
MainForm.cs
WinForms UI
LoadRoutesAsync(); BuyAsync()
selects
creates
Destination
Models.cs
Destination record
Id; Name; FareVnd
PurchaseResult
Models.cs
PurchaseResult record
OrderId; DestinationId; Quantity; TotalVnd
Method; Status; TicketCodes
persists
returns
SqlOrderStore
SqlOrderStore.cs
GetDestinationsAsync()
SavePurchaseAsync(PurchaseResult)
Source: Program.cs, MainForm.cs, Models.cs, SqlOrderStore.cs. Manual code model; not a native VS Code Map.

Visual Studio forward model-to-code evidence. The first view shows the model entities; the second shows ticket, gateway, printer, repository, and service skeletons.



ForwardModelSkeleton.cs

Miscellaneous Files | SmartTicketing.ModelSkeleton.OrderStatus | Pending

```
1  // Documentation artifact: manual model-to-code skeleton transcribed from diagrams/class.mmd.
2  // Kept outside the application project; these conceptual types are not the runnable prototype.
3  namespace SmartTicketing.ModelSkeleton;
4
5  public enum OrderStatus { Pending, Paid, Declined, ReconciliationRequired }
6  public enum PaymentMethod { Card, QrWallet }
7
8  public sealed class Destination {
9      public Guid Id { get; set; }
10     public string Name { get; set; } = string.Empty;
11     public long FareVnd { get; set; }
12     public bool Active { get; set; }
13 }
14
15 public sealed class TicketOrder {
16     public Guid Id { get; set; }
17     public Guid DestinationId { get; set; }
18     public int Quantity { get; set; }
19     public long TotalVnd { get; set; }
20     public OrderStatus Status { get; set; }
21     public string IdempotencyKey { get; set; } = string.Empty;
22     public DateTime CreatedAt { get; set; }
23     public void ConfirmPaid() => Status = OrderStatus.Paid;
24     public void Cancel() { }
25 }
26
27 public sealed class Payment {
28     public Guid Id { get; set; }
29     public Guid OrderId { get; set; }
30     public PaymentMethod Method { get; set; }
31     public string ProviderReference { get; set; } = string.Empty;
32     public long AmountVnd { get; set; }
33     public OrderStatus Status { get; set; }
34     public DateTime ExpiresAt { get; set; }
35     public void MarkPaid() => Status = OrderStatus.Paid;
36     public void MarkFailed() => Status = OrderStatus.Declined;
37 }
```

ForwardModelSkeleton.cs

Miscellaneous Files | SmartTicketing.ModelSkeleton.Ticket | IssuedAt

```
31     public string ProviderReference { get; set; } = string.Empty;
32     public long AmountVnd { get; set; }
33     public OrderStatus Status { get; set; }
34     public DateTime ExpiresAt { get; set; }
35     public void MarkPaid() => Status = OrderStatus.Paid;
36     public void MarkFailed() => Status = OrderStatus.Declined;
37 }
38
39 public sealed class Ticket {
40     public Guid Id { get; set; }
41     public Guid OrderId { get; set; }
42     public string Barcode { get; set; } = string.Empty;
43     public DateTime IssuedAt { get; set; }
44 }
45
46 public interface IPaymentGateway { Task InitiateAsync(); Task QueryStatusAsync(); Task VerifyCallbackAsync(); }
47 public interface IPrinter { Task PrintAsync(Ticket ticket); }
48 public interface IOrderRepository { Task SaveAsync(TicketOrder order); Task<TicketOrder?> GetByIdAsync(Guid id); }
49
50 public sealed class TicketService {
51     private readonly IPaymentGateway _gateway;
52     private readonly IPrinter _printer;
53     private readonly IOrderRepository _orders;
54     public TicketService(IPaymentGateway gateway, IPrinter printer, IOrderRepository orders)
55         => (_gateway, _printer, _orders) = (gateway, printer, orders);
56     public Task<TicketOrder> CreateOrderAsync() => throw new NotImplementedException();
57     public Task ConfirmPaymentAsync() => throw new NotImplementedException();
58     public Task IssueTicketsAsync() => throw new NotImplementedException();
59 }
60
```

5.1.2 Backward Engineering (Code to Model)

Backward model was produced by inspecting Program.cs, MainForm.cs, Models.cs, and SqlOrderStore.cs. The as-built model is separate from the conceptual design because the runnable prototype implements a smaller set of types.

No.	Manual code change	File / class	How it was reflected in the model
1	Added DestinationId to PurchaseResult so Declined and ReconciliationRequired orders can be saved without ticket/barcode data.	prototype/Models.cs; MainForm.cs; SqlOrderStore.cs	SqlOrderStore now uses purchase.DestinationId directly. Approved, Declined, and Unknown/timeout scenarios verify ticket creation only for Paid. The design model should be manually reviewed; no current screenshot of the refreshed model was supplied.
2	Keep payment outcome persistence independent from ticket creation; create tickets only for Paid outcomes.	prototype/SqlOrderStore.cs; MainForm.cs	The SQL test evidence shows Paid saved with tickets, while Declined and ReconciliationRequired saved without tickets. No automatic code-to-model synchronization was demonstrated.
3	Add generated QR images for simulated payment requests and successful ticket codes; send declined or unknown/timeout outcomes to the unsuccessful screen without a ticket QR.	prototype/MainForm.cs; prototype/SmartTicketing.csproj	MainForm now generates scannable QR PNGs through QRCoder 1.8.1. The refreshed source-derived model includes the QR library dependency; no Visual Studio engineering screenshot was recaptured.

Current source-derived model: prototype/engineering_evidence/CurrentCodeModel_Clear.png and BackwardCodeModel.mmd. The existing Visual Studio capture visuals/VS_BackwardModel_Fit.png is a historical view of the earlier model and predates QR rendering.

5.1.3 Code Map

The installed Visual Studio edition is Community. It cannot create native Code Maps. The refreshed source-derived dependency map is prototype/engineering_evidence/SmartTicketing_SourceDependencyMap.dgml. The existing visuals/VS_SourceDependencyMap.png is an editor capture of the earlier map, not a native Code Map and not current after QR dependency updates.

Native Code Map screenshot is unavailable in the installed Visual Studio Community edition. The preceding source-derived model is included as a clearly labeled alternative.

The source-derived dependency map summarizes dependencies among Program, MainForm, Destination, PurchaseResult, SqlOrderStore, SQL Server, QRCoder 1.8.1, and payment/status enums. It is based on current source inspection; native Code Map generation remains unavailable in Visual Studio Community.

5.2 GitHub & AI Tooling (10 marks)

5.2.1 Source Control

No GitHub repository URL, commit history, or pull-request evidence was present in the supplied folder. Add the real shared repository and verified collaboration history before submission.

Item	Your answer
Repository link	Gangvinh/Assignment1_SoftwareTechnology
Branches / collaboration evidence	No branch, commit, or pull-request history supplied.
Submission date	31st October 2026

Member	Commits	Pull requests	Main contributions
Nguyen Le Quang Vinh (524H0205)	Not supplied	Not supplied	No repository evidence supplied.
Huynh Bao Duy (524H0151)	Not supplied	Not supplied	No repository evidence supplied.
Bui Ngoc Thanh Truc (524H0201)	Not supplied	Not supplied	No repository evidence supplied.
Nguyen Gia Binh (524H0146)	Not supplied	Not supplied	No repository evidence supplied.

5.2.2 AI Assistance: Copilot Prompt Documentation

The table records actual ChatGPT prompts used during this editing work. It does not claim GitHub Copilot code generation or attribute prompts to a student.

No.	Member	Prompt	How the AI helped	Used? (kept / changed / rejected)	How we validated it
1	Unattributed user session	Can you test it for me, and	Requested test/run	Documentation support; no	Checked against report and project evidence.

		also run in Visual Studio for me?	assistance and documentation.	generated code claimed.	
2	Unattributed user session	Can you align Assignment report with bc as1?	Aligned report structure and traceability.	Document editing.	Compared section headings and requirement IDs.
3	Unattributed user session	Can you help me keeping the template in bc as1 and adding part 3 4 from Assignment report into it?	Added report-aligned Parts 3 and 4 to a copy of the brief template.	Document editing.	Reopened the copy; checked tables and mockup images.
4	Unattributed user session	Can you do part 5 in the doc file for me please?	Filled implementation, tooling, demo, and test evidence sections.	Document editing.	Compared with 524H0205_NguyenLeQuangVinh.docx and prototype source.
5	Unattributed user session	Is the part 5 correct yet? i think it still not correct	Triggered a review and correction of shifted table cells and remaining placeholders.	Document correction.	Checked each cell in the code-change and source-control tables.

5.2.3 Reflection on AI Use

In the documented editing sessions, ChatGPT was used to compare and align assignment documents and organize evidence. The supplied record does not establish which team members used Copilot or AI to generate application code.

What worked:

AI assistance helped organize requirements, use-case traceability, and report/workbook content. Runtime and database behavior were checked against the available project and test evidence.

What did not work:

AI-generated text can contain assumptions or mismatch the assignment. The team should check claims against the brief and implementation; this log does not claim AI-generated code.

Errors found in AI output:

Testing exposed a persistence defect for declined and unknown outcomes because they have no ticket barcode. PurchaseResult was updated to carry DestinationId, and persistence was corrected. This is supported by the report test evidence and is not attributed here to AI code generation.

What we would do differently:

For future work, record prompts and responsible members when used, verify suggestions against requirements, and test in the target environment.

AI supported document preparation in this record. Human review and execution evidence remain necessary; the Visual Studio run and SQL outcomes are documented separately.

5.3 Demo Application (10 marks)

The supplied prototype is a C# Windows Forms TVM demo backed by SQL Server. It simulates card/QR payment outcomes and does not connect to a live provider or physical kiosk hardware.

Item	Your answer
Technology	C# Windows Forms; .NET 8.0-windows with major roll-forward; Microsoft.Data.SqlClient 6.1.2; QRCoder 1.8.1 for scannable payment-demo and ticket QR images.
IDE / runtime test	Visual Studio Community 18; .NET 10.0.12 runtime; tested 31 October 2026.
Database	SQL Server LocalDB database SmartTicketing_VSRun; schema.sql then seed.sql. By default the app connects to (localdb)\MSSQLLocalDB using Windows authentication.
Run / demo	Open prototype/SmartTicketing.slnx or .csproj and press F5. The app defaults to LocalDB SmartTicketing_VSRun; run schema.sql then seed.sql if the tables are missing. Select a route and quantity 1–10, choose card or QR, and select a simulated outcome. Set SMARTTICKETING_SQL to override the connection; set it to offline for an in-memory demo that does not save purchases.

Input form

The prototype form provides a destination selector, an editable quantity control from 1 to 10, card or QR selection, and a simulated gateway outcome selector. A runtime screenshot was not supplied.

Output report or success screen

The successful result displays a QR encoding the ticket code. Declined and unknown/timeout results display an unsuccessful message with no ticket QR. A runtime screenshot was not supplied; SQL-backed scenarios are listed below.

Test scenarios

No.	Scenario	Input	Expected result	Result
1	Approved purchase	Card (simulated), quantity 1, Approved	Persist Paid order/payment and one ticket.	Pass: paid state saved; ticket count matched quantity.
2	Payment declined	Card (simulated), quantity 1, Declined	Persist declined order/payment; issue no ticket.	Pass: Declined saved; zero tickets.
3	Unknown / timeout	Card (simulated), quantity 1, Unknown / timeout	Persist unresolved/reconciliation state; issue no ticket.	Pass: ReconciliationRequired saved; zero tickets.

Known limitations

The payment QR is a scannable demo payload only; no bank or wallet API is connected. The ticket QR encodes the prototype ticket code, but no physical printer or gate validation service is connected. The prototype supports destination and quantity only; ticket-type and passenger-discount requirements FR-02/FR-03 are not implemented. The UI is English-only, so the bilingual FR-16 requirement is not implemented. Visual Studio forward/backward engineering, Code Map evidence, and GitHub history

were not supplied. Production security, PCI compliance, and performance targets are not established by this local demo.

Payment is simulated; no live bank/wallet API, card reader, or printer is connected. UML forward/backward engineering, Code Map, and GitHub history evidence were not supplied. The local test does not establish production security, PCI compliance, or performance targets.

6. Checklist for "LabGroup_QT2_Assignment1.pdf"

Guidance (delete before submitting): *The submission must address the content of LabGroupAssignment1.pdf (initial context and submission guidelines). List each item of that document, where this report covers it, and tick it.*

No.	Item from LabGroupAssignment1.pdf	Where addressed in this report	Done
1	Clarification Q&A	Section 2.1 – Clarification Q&A	X
2	Requirements Specification	Section 2.2 – Requirements Specification	X
3	Use Case Modeling	Section 3.1 – Use Case Modeling	X
4	Activity Diagrams	Section 3.2 – Activity Diagrams	X
5	Detailed System Modeling: "Buy a Ticket"	Section 3.3 – Detailed System Modeling	X
6	UI/UX Design	Section 4.1 – UI/UX Design	X
7	Architectural Design	Sections 4.2.1 and 4.2.2 – Architecture Diagram and Deployment Diagram	X
8	Visual Studio Engineering (Model-to-Code)	Section 5.1 – Visual Studio Engineering	X
9	GitHub & AI Tooling (Copilot)	Section 5.2 – GitHub & AI Tooling	X
10	Demo Application (C# and MSSQL)	Section 5.3 – Demo Application	X

7. Contribution of Each Member

Guidance (delete before submitting): *The % contribution of each member must total 100% and match readme.txt.*

Member	Student ID	Work products and tasks	% contribution
Nguyễn Lê Quang Vinh	524H0205	Clarification Q&A, Requirements Specification, and Use Case Modeling.	25%
Huỳnh Bảo Duy	524H0151	Activity Diagrams, Sequence Diagram, State Chart Diagram, and Class Diagram.	25%
Nguyễn Gia Bình	524H0146	UI/UX Design, Architecture Diagram, Deployment Diagram, and Visual Studio Engineering.	25%
Bùi Ngọc Thanh Trúc	524H0201	GitHub management, AI Prompt Documentation, Demo Application documentation, and final report.	25%
Total			100%

8. Conclusion and Lessons Learnt

8.1 Conclusion

Through this project, our team gained practical experience in analyzing, designing, and developing a Ticket Vending Machine (TVM) system. We applied software engineering knowledge to create system diagrams, design the system architecture, and develop a demo application using C# and Microsoft SQL Server.

The project helped us understand how ticket management, payment processing, and database persistence work together. The prototype was exercised in Visual Studio; the supplied folder does not include GitHub collaboration evidence.

However, the project still has some limitations. The demo application does not fully integrate with real payment gateways or physical vending machine hardware. Some functions are simulated for demonstration purposes. In addition, the system may need further improvements in security, error handling, and user interface design before being used in a real environment.

8.2 Lessons Learnt

During the project, each team member gained different knowledge and experience based on their assigned tasks. Overall, we improved our skills in system analysis, UML modeling, C# programming, database design, and teamwork.

The assignment highlights the need to record source-control collaboration and AI use as they occur. AI-generated suggestions should be reviewed and tested before they are applied.

In conclusion, this project provided valuable experience in applying theoretical knowledge to a practical software development project. It also helped our team improve both technical skills and collaboration skills.

References

1. Object Management Group (OMG). (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1
2. Microsoft. (n.d.). *C# Documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/
3. Microsoft. (n.d.). *SQL Server Documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/sql/sql-server/
4. Microsoft. (n.d.). *Visual Studio Documentation*. Microsoft Learn. https://learn.microsoft.com/en-us/visualstudio/
5. GitHub. (n.d.). *GitHub Docs*. https://docs.github.com/
6. Figma. (n.d.). *Figma Help Center*. https://help.figma.com/
7. National Payment Corporation of Vietnam (NAPAS). (n.d.). *NAPAS – National Payment Services*. https://napas.com.vn/

Appendix

Appendix A: readme.txt

[Paste the contents of readme.txt: GitHub link, Figma link, database script instructions, % contribution]

Appendix B: Full-size Diagrams

[Full-size use case, activity, sequence, state, class, architecture and deployment diagrams]